2026 IEEE/ACM International Symposium on Quality of Service (IWQoS) | 979-8-3195-1703-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/IWQoS70441.2026.11661019

Odysseus: A Context-Level Pre-training Framework for Out-of-Distribution Encrypted Traffic Classification

Wenqi Dong[1,2], Xinjie Lin[3], Longtao He[4,*], Gaopeng Gou[1,2], Zhen Li[1,2],
Junzheng Shi[1,2], Jianshuo Liu[5], Gang Xiong[1,2]
[1]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3]Zhongguancun Laboratory, Beijing, China
[4]National Computer Network Emergency Response Technical Team / Coordination Center of China, Beijing, China
[5]University of Michigan, Ann Arbor, MI, USA
{dongwenqi, gougaopeng, lizhen, shijunzheng, xionggang}@iie.ac.cn;
linxj@mail.zgclab.edu.cn; hlt@cert.org.cn; jianshuo@umich.edu

***Abstract*—Encrypted traffic classification is essential for network security, management, and Quality of Service (QoS), but models trained under IID assumptions often degrade after deployment due to application updates, temporal drift, and infrastructural changes. We observe that short-window multi-flow context provides more stable behavioral cues than isolated flows, while self-supervised pre-training can learn transferable priors from unlabeled traffic without target-domain labels. We present Odysseus, a context-level pre-training framework for OOD encrypted traffic classification. Odysseus converts raw pcaps into a Multi-Scale Flow Grid Representation (MSFGR) and pre-trains a masked autoencoder with complementary packet- and flow-level masking to capture intra-/inter-flow structure. During fine-tuning, it integrates Hierarchical Attention Distillation (HAD), Traffic-Aware Augmentation and Contrastive Learning (TACL), and Associated Flow Prediction (AFP) to enhance context modeling, distill context knowledge into a deployable flow-level branch, and regularize OOD-robust relational representations. Across 11 scenarios (5 IID, 6 OOD) spanning Cross-Version, Cross-Time, Cross-Spatiotemporal, and Cross-Type shifts, Odysseus achieves state-of-the-art performance and improves OOD F1-Score by up to 15.70%, reaching 85.38%, 90.05%, 71.93%, and 99.62/96.87/72.55% F1 on the corresponding OOD settings. Further analyses confirm the effectiveness of each component, strong few-shot robustness, graceful degradation under noisy/missing context, and advantages over matrix-based inputs.**

***Index Terms*—Encrypted traffic classification, out-of-distribution generalization, pre-training, multi-flow representation.**

I. Introduction

ENCRYPTED network traffic has become the dominant carrier of modern Internet services, driven by the ubiquitous adoption of Transport Layer Security (TLS) and QUIC transport protocol. While this encryption crucially protects user privacy and data integrity, its inherent opacity severely complicates traditional traffic classification [1], [2]. Without payload visibility, both application identification and threat detection have become critical bottlenecks for network administrators [3], [4]. Consequently, advanced encrypted traffic analysis has emerged as an indispensable capability, providing the foundation for effective network management, stringent cybersecurity, and dynamic Quality of Service (QoS) delivery.

A major obstacle to practical deployment, however, is *out-of-distribution (OOD) generalization*. In real-world operational environments, the traffic distributions of target applications shift continuously after deployment: application versions update frequently, communication stacks and APIs vary across platforms, CDNs and network policies change, and adversaries continuously evolve traffic fingerprints [5]–[7]. These shifts can alter superficial traffic statistics in ways that are difficult to anticipate, causing models developed under independent and identically distributed (IID) assumptions to fail silently in the field [8], [9]. This issue is particularly acute for encrypted traffic, where the observable feature space is inherently limited and thus more vulnerable to spurious correlations [10].

Inspired by frameworks such as BERT [11] and MAE [12], recent progress in self-supervised pre-training suggests a promising direction: by learning generic representations from large-scale unlabeled traffic, a model can reduce dependence on extensive labeled data and transfer more effectively to downstream tasks [13]–[15]. Nonetheless, existing pre-training pipelines for encrypted traffic typically operate on *single flows* and therefore inherit a key limitation: they under-utilize the interaction structure of real communication, where applications often generate *multiple concurrent flows* within short time windows to improve efficiency and satisfy shared objectives [16], [17]. Such multi-flow behaviors tend to remain stable across distributions because they are governed by application logic rather than transient network conditions, making inter-flow relationships a valuable source of invariance [18]–[20]. Compared with

*Longtao He is the corresponding author.

979-8-3195-1703-6/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

marginal per-flow features (e.g., packet length and inter-arrival distributions) that can drift with network conditions, short-window co-occurrence and ordering patterns across concurrent flows are largely tied to application logic. Indeed, passively captured traffic frequently exhibits synchronized handshakes and correlated transmissions across flows, while prior methods focusing on per-flow inputs fail to exploit these dependencies [21].

This paper is motivated by two observations. First, **multi-flow context** provides more stable behavioral cues than isolated single-flow patterns, offering an opportunity to mitigate distribution-dependent noise. Concretely, we focus on *anchor-centered short-window context* that aggregates a small set of temporally adjacent flows, which is lightweight to construct and more stable than isolated flows [19], [20]. Second, **self-supervised pre-training** can learn transferable and robust priors from abundant unlabeled traffic, improving generalization to shifted deployments even when training and model selection use only source-distribution data. Leveraging both, we present Odysseus, a context-level pre-training framework for OOD encrypted traffic classification.

Odysseus is built on a new representation, the **Multi-Scale Flow Grid Representation (MSFGR)**, which converts raw pcaps into structured matrices that encode packet-level side-channel features together with short-window inter-flow organization. On top of MSFGR, Odysseus follows a two-stage learning paradigm. In the *pre-training stage*, we design a tailored masked autoencoding objective with complementary packet-level masking to capture fine-grained transport patterns and flow-level masking to enforce reasoning over inter-flow structure. This hybrid masking exposes the model to both local fidelity and global context completion, allowing it to learn **intra-flow** and **inter-flow correlations** in an unbiased, label-free manner. In the *fine-tuning stage*, we introduce three components that jointly improve robustness under distribution shift: (i) **Hierarchical Attention Distillation (HAD)** hierarchically models intra-flow and inter-flow dependencies and *distills* context-enriched multi-flow knowledge into a single-flow branch, enabling strong performance even when multi-flow context is unavailable at inference time, (ii) **Traffic-Aware Augmentation and Contrastive Learning (TACL)** applies semantics-preserving, protocol-consistent perturbations and a weighted contrastive objective to enforce invariance to realistic network variability, and (iii) **Associated Flow Prediction (AFP)** trains the model to detect semantic inconsistency under inter-flow perturbations, strengthening its reliance on label-relevant structure rather than distribution-specific features.

We summarize our main contributions as follows:

1) We propose Odysseus, a context-level pre-training framework for OOD encrypted traffic classification, leveraging temporal-related multi-flow behavioral constancy and unlabeled pre-training to improve cross-distribution robustness.
2) We introduce MSFGR, a structured representation that jointly captures intra-flow side channels and inter-flow organization, and design a pre-training strategy with packet-level and flow-level masking to learn transferable traffic priors without labels.
3) We develop a fine-tuning suite comprising HAD, TACL, and AFP, where hierarchical modeling and knowledge distillation enable practical single-flow deployment while contrastive invariance and inter-flow consistency learning further enhance OOD robustness.
4) Extensive experiments over 11 scenarios (5 IID, 6 OOD) spanning Cross-Version, Cross-Time, Cross-Spatiotemporal, and Cross-Type shifts demonstrate that Odysseus achieves state-of-the-art performance and improves OOD F1-Score by up to 15.70% (e.g., 85.38% on Cross-Version, 90.05% on Cross-Time, 71.93% on Cross-Spatiotemporal, and 99.62/96.87/72.55% across Cross-Type scenarios), with additional analyses in few-shot learning and robustness evaluations.

II. Related Work

Encrypted Traffic Classification. Prior encrypted traffic classification methods include rule-based inspection (e.g., DPI), feature-engineered machine learning [1], [22], [23], and deep models that learn end-to-end flow/packet representations [4], [24]–[26]. While these approaches can perform well under IID assumptions, their decision cues may rely on dataset-specific packetization/timing features and degrade under version/time/environment shifts. Recently, pre-training on large-scale unlabeled traffic has been explored to learn transferable priors and reduce label dependence [3], [13]–[15]. However, most pre-training pipelines remain flow-centric, and existing context-aware classifiers typically lack context-level pre-training and context-to-flow distillation for robust deployment. For instance, FG-Net and TrafficScope leverage multi-flow context to boost performance, but do not incorporate a pre-training architecture to further improve performance [16], [17].

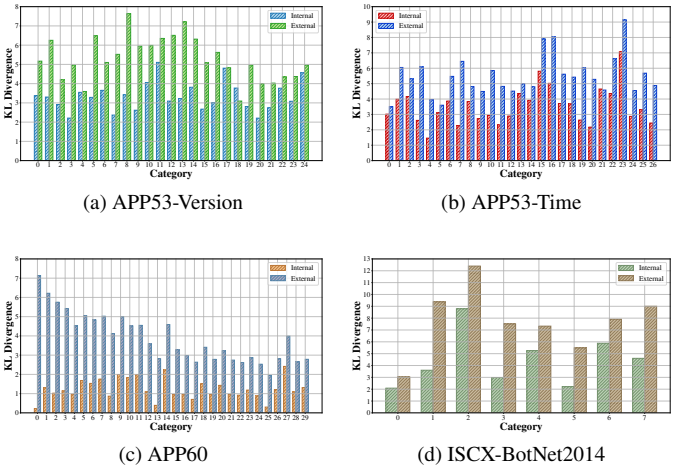
OOD Solutions. For distribution shift, practical traffic systems often resort to retraining or incremental updates (e.g., BIND, AdaWFP) [27], [28] or partial renewal (e.g., TrAdaBoost) [29], [30], which are costly and require new data. Unsupervised adaptation (e.g., FDAN) further assumes access to unlabeled target traffic during training [9]. Inspired by these directions, Odysseus pursues OOD robustness for encrypted traffic by exploiting stable inter-flow association and distilling context-enriched knowledge into a deployable single-flow branch. It avoids reliance on target-distribution information and reduces dependence on scenario-specific assumptions often made in prior work, such as multiple source environments [5], the assumption that the target distribution must be capable of forming context and extra deployment overhead [18], or specialization to malicious families [21].

III. Design Goals and Challenges

A. Design Goals

We aim to build an encrypted traffic classification system, Odysseus, that remains reliable under *OOD deployments*—covering encrypted application identification and mali-

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.



(a) APP53-Version (b) APP53-Time

(c) APP60 (d) ISCX-BotNet2014

Fig. 1. Internal and external distribution differences between classes.

cious traffic detection—when payload inspection is unavailable and traffic features drift over time.

Formally, let the labeled source data be $D_S = \{(x_i, y_i)\}_{i=1}^{N} \sim P_S(X, Y)$ and the deployment-time data follow $P_T(X, Y)$ with the same label space. Here, x_i denotes an *anchor-centered multi-flow* ***context*** *instance* constructed from raw traces and encoded as a fixed-size MSFGR tensor (§IV-B), consisting of an *anchor flow* and its temporally adjacent *neighbor flows*, and y_i is the class label of the anchor flow. Our goal is to learn parameters θ from D_S such that the expected target risk $\mathbb{E}_{(x,y)\sim P_T}[\mathcal{L}(\mathcal{M}_\theta(x), y)]$ is minimized, while model development does not rely on any target-distribution labels or target-specific adaptation.

In practice, $P_T \neq P_S$ arises from software evolution (e.g., version updates), temporal drift, and heterogeneous network stacks/policies, which jointly reshape side-channel statistics and transport patterns. To illustrate the severity of such shifts, we measure KL divergence [31] between *internal-span* (IID vs. IID) and *external-span* (IID vs. OOD) distributions on the payload-free statistical features defined in [22].[1] As shown in Fig. 1, external-span divergence is larger for most classes, indicating that realistic deployment shifts can invalidate models developed under standard IID assumptions.

B. Challenges

OOD generalization for encrypted traffic is difficult because only payload-free side channels are observable and their statistics can drift substantially across deployments [5]–[7], [9], [18], [34]. Such shifts are often class-conditional and dominated by superficial marginal changes, making ERM prone to learning unstable shortcuts tied to source capture conditions [10]. Robustness regularization must therefore be semantics-preserving: naive perturbations may violate protocol/causality or leak labels [35], while effective self-supervision should encourage learning stable functional logic rather than brittle artifacts [36].

[1]We report the comparison on four public datasets under OOD settings: App53-Version, App53-Time [16], APP60 [32] (30 classes sampled for KL visualization in Fig. 1, while our experiments use the APP60 protocol), and ISCX-BotNet2014 [33].

Finally, anchor-centered short-window *contexts* can provide stable interaction evidence [16], [17], [19], [20], yet neighbor flows may be missing or noisy at deployment, requiring a context-aware model with a strong single-flow fallback.

These challenges motivate Odysseus to couple MSFGR, label-free dual-masked pre-training, and shift-aware fine-tuning with context-to-flow transfer (HAD/TACL/AFP).

IV. Methodology

A. Framework

Fig. 2 presents Odysseus. We first encode anchor-centered short-window context into a **Multi-Scale Flow Grid Representation (MSFGR)**. We then pre-train a dual-masked **Masked Autoencoder (MAE)** on unlabeled MSFGR to learn transferable priors. Finally, we fine-tune with **Hierarchical Attention Distillation (HAD)**, **Traffic-Aware Augmentation and Contrastive Learning (TACL)**, and **Associated Flow Prediction (AFP)** to enable context inference and a deployable flow-level branch.

B. Data Processing Stage

We convert each pcap trace of specific class into **MSFGR** that couples *intra-flow* and *inter-flow* associations. The same pipeline constructs unlabeled (pre-training), IID source (fine-tuning), and OOD target (evaluation-only) splits.

Packet Gathering → Packet Sequence. Given a pcap trace X_i, we group forward and reverse packets into bidirectional flows based on the five-tuple. For each flow, we build a packet sequence with packet length (Len), relative timestamp (Time, offset to the first packet), and direction (Dire).

Flow Grouping. We sort flows by start time and group them with a sliding window of five flows and stride 1, denoted as $\{F_0, \ldots, F_4\}$. We treat F_2 as the *anchor flow* and $\{F_0, F_1, F_3, F_4\}$ as *neighbor flows* (two preceding and two subsequent flows). We use a 5-flow window to provide symmetric short-term context around the anchor while keeping the input compact; larger windows may introduce unrelated flows and increase memory/latency. If a trace boundary yields an incomplete window, we duplicate the closest available flows to maintain a fixed 5-flow input. Thus, every flow can be used once as the anchor, ensuring full input-flow coverage.

Matrix Generation → Patchify. Each 5-flow group is mapped to a 3-channel 40×40 tensor $X_i^* \in \mathbb{R}^{3\times 40\times 40}$, where channels C_0, C_1, and C_2 encode Len, Time, and Dire. The matrix is vertically split into five 8-row segments, one per flow. Within each segment, packets are placed into non-overlapping 2×2 patches in row-major order, with each packet filling its patch using the same Len/Time/Dire value. This yields P_f=80 patches per flow and L=400 patches per MSFGR. This cap captures early-flow behavior while bounding computation; longer flows are truncated and shorter flows are zero-padded.

We encode Len in [40, 1520] with 0 for empty patches, Time as per-flow elapsed time truncated to [0, 20] seconds with T_0=0, and Dire as +1 for forward and −1 for backward. Empty patches use Len= 0 and Time= 0. The label y_i is inherited from the anchor flow.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

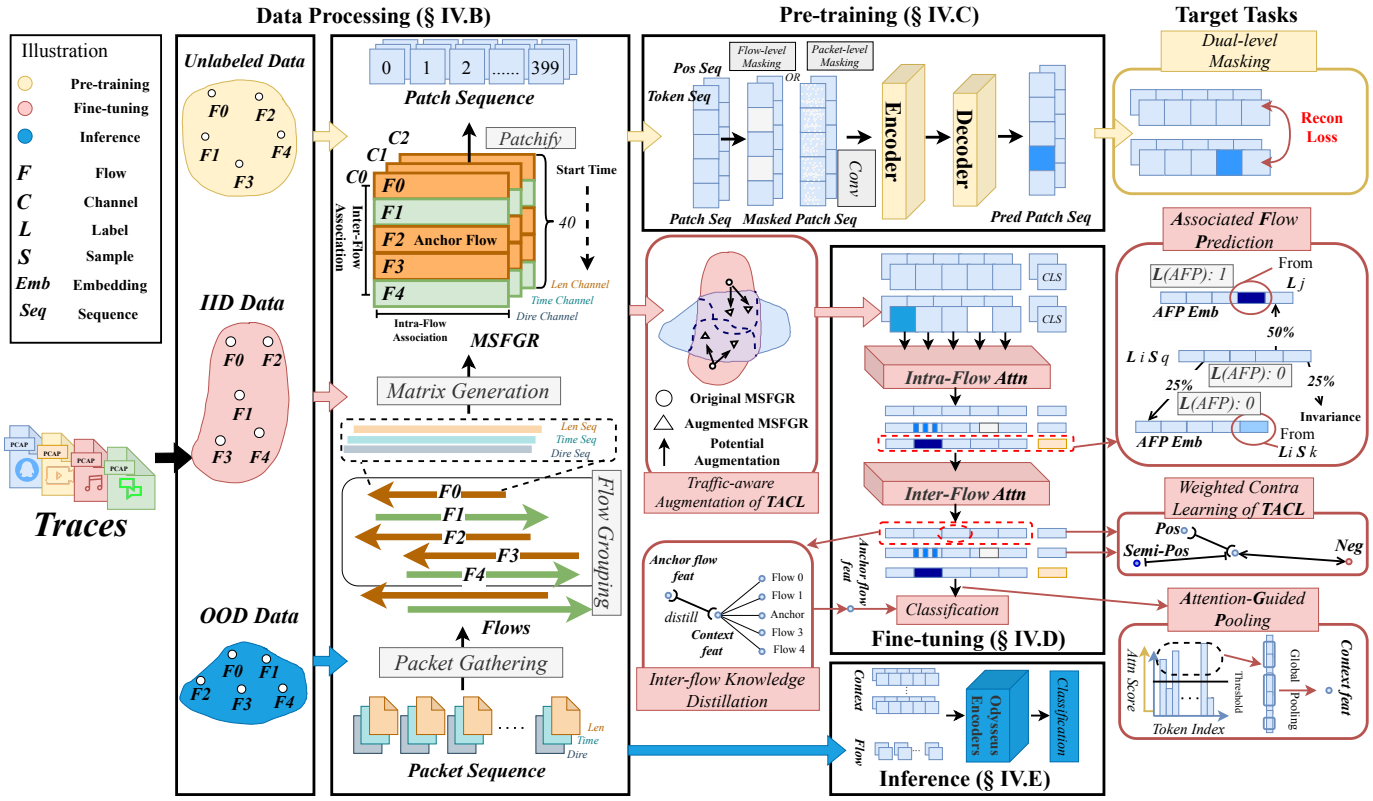


Fig. 2. Overview of Odysseus framework.

C. Pre-training Stage

We pre-train the MAE on unlabeled MSFGR to capture *packet-level* regularities and *flow-level* semantics. The L=400 patch tokens (§IV-B) are projected to 192-D embeddings by a convolutional layer, augmented with learnable positional embeddings, and fed into a ViT-style MAE with a 4-block encoder (16 heads, MLP dim 768) and a 2-block decoder [12].

Dual-level Masking. To provide complementary signals at packet and flow granularity, we apply a hybrid masking strategy. For each sample, we choose one scheme with probability 0.5.

(i) *Packet-level masking* randomly masks a proportion r of the L patches. We sample $\lfloor rL \rfloor$ indices without replacement and define $\mathbf{m}^{\text{pkt}} \in \{0,1\}^L$:

$$m_i^{\text{pkt}} = \begin{cases} 1, & \text{if } i \in \mathcal{R}, \\ 0, & \text{otherwise,} \end{cases} \qquad i = 0, \ldots, L-1, \tag{1}$$

where $\mathcal{R}$ is a uniformly random subset of $\{0, 1, \ldots, L-1\}$ with $|\mathcal{R}| = \lfloor rL \rfloor$. We set $r = 0.9$ following [15].

(ii) *Flow-level masking* masks entire flows. Each MSFGR contains $N_f = 5$ flows with $P_f = 80$ patches per flow, so $L = N_f P_f$. We uniformly sample two flows to be fully masked. Let $\mathcal{S} \subseteq \{0, 1, 2, 3, 4\}$ be the masked flow indices with $|\mathcal{S}| = 2$. During pre-training, flows are treated symmetrically, so masking may select any flow, including the anchor. The binary mask $\mathbf{m}^{\text{flow}} \in \{0,1\}^L$ is

$$m_i^{\text{flow}} = \begin{cases} 1, & \text{if } \lfloor i/P_f \rfloor \in \mathcal{S}, \\ 0, & \text{otherwise,} \end{cases} \qquad i = 0, \ldots, L-1, \tag{2}$$

where $m_i^{\text{flow}} = 1$ indicates that the i-th patch is masked.

Let $\mathbf{m} \in \{\mathbf{m}^{\text{pkt}}, \mathbf{m}^{\text{flow}}\}$ be the mask used for the current sample and let $\mathcal{M} = \{k \mid m_k = 1\}$ be the masked index set. The decoder reconstructs masked patches from the encoder output and we optimize the mean squared error:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} \|\mathbf{v}_k - \hat{\mathbf{v}}_k\|_2^2. \tag{3}$$

where $\mathbf{v}_k$ and $\hat{\mathbf{v}}_k$ are the original and reconstructed patch values.

D. Fine-tuning Stage

1) Overview: We fine-tune the pre-trained encoder with three components: **HAD**, **TACL**, and **AFP**.

2) HAD: HAD performs intra-flow encoding, inter-flow contextualization, and context-level-to-single-flow distillation.

a) Intra-Flow Attention: We keep the anchor flow unchanged. For each neighbor flow, we apply TACL with probability 0.5 to form an augmented view (§IV-D3). Let the five flows be indexed as $t \in \{j, \ldots, j+4\}$ with the anchor at $t = j+2$.

Each flow is tokenized into patches and projected to 192-D embeddings. We prepend a learnable [CLS] token, add positional embeddings, and compute intra-flow self-attention with the shared 4-block Transformer encoder backbone from pre-training stage:

$$\mathcal{H}_t = \text{MSA}(emb_t + pos_t), \quad t \in \{j, \ldots, j+4\}, \tag{4}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

where $\mathrm{MSA}(\cdot)$ denotes the shared 4-block encoder backbone. We use the intra-flow [CLS] token $\mathbf{c}_t$ as the flow summary and mean-pool every 4 consecutive non-CLS patch tokens (80→20):

$$emb_t^r = \mathrm{Pool}(\mathcal{H}_t^{\mathrm{patch}}) \in \mathbb{R}^{20\times 192}, \quad t \in \{j, \ldots, j+4\}. \tag{5}$$

We cache the anchor summary $\mathbf{c}_{\mathrm{anc}} = \mathbf{c}_{j+2}$ for distillation.

b) Inter-Flow Attention: We build three inter-flow inputs: the original x_o, a TACL-augmented view x_{aug}, and an AFP-perturbed view x_a with label y_a (§IV-D4). Each input x concatenates the pooled tokens $\{emb_t^r\}_{t=j}^{j+4}$ and prepends an inter-flow [CLS] initialized by averaging the intra-flow summaries $\{\mathbf{c}_t\}_{t=j}^{j+4}$. A shared attention encoder produces contextualized sequences x'_o, x'_{aug}, and x'_a:

$$x' = \mathrm{MSA}(x + pos). \tag{6}$$

Here, $\mathrm{MSA}(\cdot)$ denotes the shared encoder backbone across the intra-flow and inter-flow stages.

We summarize x'_o and x'_a with attention-guided pooling: we average the [CLS] token and the top-K tokens ranked by their [CLS]-attention weights in the last inter-flow layer, where K=20. This yields $z_{x'_o}$ and $z_{x'_a}$. For x'_{aug}, we use global average pooling to obtain $z_{x'_{\mathrm{aug}}}$.

We apply a multi-class head on $z_{x'_o}$ with label y, a binary AFP head on $z_{x'_a}$ with label y_a, and a contrastive objective between $z_{x'_{\mathrm{aug}}}$ and $z_{x'_o}$.

c) Inter-Flow Knowledge Distillation: We distill the context-level representation into a single-flow branch for deployment. We treat the context summary $z_{x'_o}$ as the teacher:

$$\mathbf{f}_{\mathrm{multi}} = \mathrm{Norm}(z_{x'_o}) \in \mathbb{R}^d, \tag{7}$$

where $\mathrm{Norm}(\cdot)$ denotes ℓ_2 normalization. For the anchor flow, we compute the student feature from the cached anchor summary:

$$\mathbf{f}_{\mathrm{single}} = \mathrm{Norm}(\mathbf{c}_{\mathrm{anc}}) \in \mathbb{R}^d. \tag{8}$$

We align teacher and student with a cosine distillation loss, detaching $\mathbf{f}_{\mathrm{multi}}$:

$$\mathcal{L}_{\mathrm{distill}} = 1 - \frac{\mathbf{f}_{\mathrm{multi}} \cdot \mathbf{f}_{\mathrm{single}}}{\|\mathbf{f}_{\mathrm{multi}}\| \cdot \|\mathbf{f}_{\mathrm{single}}\|}, \tag{9}$$

and add an auxiliary single-flow classification loss:

$$\mathcal{L}_{\mathrm{single}} = \mathrm{CrossEntropy}(\mathrm{Head}_{cls}(\mathbf{f}_{\mathrm{single}}), y). \tag{10}$$

We share the same classification head Head_{cls} for both $z_{x'_o}$ and $\mathbf{f}_{\mathrm{single}}$.

Overall fine-tuning objective. We optimize a weighted sum of the context-level classification loss, the single-flow auxiliary loss, the distillation loss, and two robustness regularizers:

$$\mathcal{L} = \mathcal{L}_{\mathrm{multi}} + 0.5\,\mathcal{L}_{\mathrm{single}} + \mathcal{L}_{\mathrm{distill}} + 0.3\,\mathcal{L}_{\mathrm{TACL}} + 0.5\,\mathcal{L}_{\mathrm{AFP}}. \tag{11}$$

where the loss weights are selected based on source-validation performance and kept fixed across datasets to avoid target leakage. The main context-level supervision is

$$\mathcal{L}_{\mathrm{multi}} = \mathrm{CE}\big(\mathrm{Head}_{cls}(z_{x'_o}),\, y\big). \tag{12}$$

where $\mathrm{Head}_{cls}(\cdot)$ is a lightweight linear classification head.

3) TACL:

a) Traffic-aware augmentations: For each neighbor flow selected for augmentation in §IV-D2, we operate on its patch-token sequence $\{q_i\}_{i=0}^{P_f-1}$ with P_f=80, where $q_i = (\ell_i, T_i, D_i)$ denotes packet length, timestamp, and direction at position i. To simulate realistic network perturbations, we sample one operator from 6 choices using approximately normalized weights $\mathbf{w} = [0.176, 0.176, 0.176, 0.176, 0.176, 0.118]$.

We define a small-packet index set over non-empty positions:

$$\xi = Q_{0.2}(\{\ell_i \mid \ell_i > 0\}), \qquad C = \{i \mid 0 < \ell_i < \xi\}. \tag{13}$$

where $Q_{0.2}(\cdot)$ denotes the 20th percentile.

Piecewise Time Warp. To simulate network jitter, we split the index range into three segments and rescale inter-arrival times by $\alpha_k \sim \mathcal{U}(0.9, 1.2)$ while keeping timestamps non-decreasing:

$$\begin{aligned} (b_0, \ldots, b_3) &= (0, \lfloor 0.3P_f \rfloor, \lfloor 0.7P_f \rfloor, P_f), \\ \Delta T_i &= T_i - T_{i-1} \quad (i \geq 1), \qquad T'_0 = T_0, \\ T'_i &= \min\Big(20,\ T'_{i-1} + \alpha_{k(i)} \Delta T_i\Big), \quad i \geq 1, \\ k(i) &= \arg\max_{k\in\{0,1,2\}} \mathbf{1}[\, b_k \leq i < b_{k+1}\,]. \end{aligned} \tag{14}$$

Micro-burst Reorder. With probability $\rho_{\mathrm{swap}} = 0.2$, perform $K \sim \mathcal{U}\{1, 3\}$ adjacent swaps while keeping time unchanged:

$$\begin{aligned} &\text{for } r = 1, \ldots, K: \quad j_r \sim \mathcal{U}\{0, P_f - 2\}, \\ &\qquad (\ell_{j_r}, D_{j_r}) \leftrightarrow (\ell_{j_r+1}, D_{j_r+1}),\ T' = T. \end{aligned} \tag{15}$$

Packet Duplication. To simulate retransmissions ($\rho_{\mathrm{dup}} = 0.3$), we duplicate $K \sim \mathcal{U}\{3, 4\}$ packets and overwrite their immediate successors. To preserve temporal causality, the duplicated packet is assigned a timestamp within its neighboring temporal interval, and the whole sequence is monotonically repaired after each update:

$$\begin{aligned} &\text{for } r = 1, \ldots, K: \quad j_r \sim \mathcal{U}\{0, P_f - 2\}, \\ &\qquad (\ell'_{j_r+1}, D'_{j_r+1}) \leftarrow (\ell'_{j_r}, D'_{j_r}), \\ &\qquad T'_{j_r+1} \leftarrow \begin{cases} \frac{T'_{j_r} + T'_{j_r+2}}{2}, & j_r + 2 < P_f, \\ T'_{j_r}, & j_r + 2 = P_f, \end{cases} \\ &\qquad T'_i \leftarrow \max(T'_i, T'_{i-1}), \quad i = 1, \ldots, P_f - 1. \end{aligned} \tag{16}$$

Silence Injection. Add a uniform delay after the middle point:

$$T'_i = \min(20,\ T_i + \delta), \quad \forall i \geq \lfloor P_f/2 \rfloor, \quad \delta \sim \mathcal{U}(0.05, 0.15). \tag{17}$$

Direction Jitter. Pick up to three non-empty, non-edge small packets and flip their directions:

$$\begin{aligned} &\mathcal{I}_D = \mathrm{sample}(C \cap [1, P_f - 2],\ \min(3, |C \cap [1, P_f - 2]|)), \\ &D'_i = -D_i, \quad i \in \mathcal{I}_D. \end{aligned} \tag{18}$$

If $\mathcal{I}_D = \emptyset$, this operator is skipped.

Length Dithering. Add bounded Gaussian perturbation to up to three packets:

$$\begin{aligned} b &= \min(0.1\bar{\ell}, 90), \qquad \sigma = 0.05\,\bar{\ell}, \\ \epsilon_i &\sim \mathcal{N}(0, \sigma^2), \qquad i \in \mathcal{I},\ |\mathcal{I}| \sim \mathcal{U}\{1, 3\}, \\ \ell'_i &= \mathrm{clip}\Big(\ell_i + \mathrm{clip}(\epsilon_i, -b, b),\ 40,\ 1520\Big). \end{aligned} \tag{19}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

where $\bar{\ell}$ is the mean packet length of the flow and $\mathcal{I}$ is a uniformly sampled index set.

b) Weighted contrastive objective: For the i-th sample, we set $u_i = \mathrm{Norm}(z^{(i)}_{x'_o})$ and $u_{i^+} = \mathrm{Norm}(z^{(i)}_{x'_{\mathrm{aug}}})$ (§IV-D2). For anchor u_i, let u_{i^+} be its positive; within-batch, $P(i)$ are same-label samples excluding $\{i, i^+\}$, and $N(i)$ are different-label samples. Using cosine similarity $s_{i,j} = \mathrm{sim}(u_i, u_j) = u_i^\top u_j$, we define

$$\mathcal{L}_{\mathrm{TACL}} = -\log \frac{e^{s_{i,i^+}/t} + \alpha \sum_{j\in P(i)} e^{s_{i,j}/t}}{e^{s_{i,i^+}/t} + \alpha \sum_{j\in P(i)} e^{s_{i,j}/t} + \sum_{k\in N(i)} e^{s_{i,k}/t}}. \tag{20}$$

where t=0.1 is the temperature and $\alpha = 0.5$ weights semi-positives. This objective enforces invariance yet maintains separation.

4) AFP: To improve OOD generalization when inter-flow patterns are disturbed, AFP supervises inter-flow semantic consistency.

Starting from the inter-flow input x_o, we stochastically construct a perturbed variant x_a and its binary label y_a:

- **Different-label substitution (50%):** replace one neighbor flow with a flow from a different class, and set $y_a = 1$ to mark semantic inconsistency.
- **No modification (25%):** keep all flows unchanged and set $y_a = 0$.
- **Same-label substitution (25%):** replace one neighbor flow with a flow from the same class and set $y_a = 0$ to tolerate intra-class variation.

Using the summary vector $z_{x'_a}$ and the binary head defined in §IV-D2, we predict $\hat{y}_a \in (0, 1)$. The AFP loss is

$$\mathcal{L}_{\mathrm{AFP}} = -\left[y_a \log(\hat{y}_a) + (1 - y_a)\log(1 - \hat{y}_a)\right]. \tag{21}$$

AFP regularizes representation learning toward domain-invariant relational cues by explicitly penalizing semantic inconsistency under perturbed flow compositions.

E. Inference Stage

At deployment, we construct MSFGR and infer with shared weights in two modes. **Context inference** uses an anchor-centered 5-flow MSFGR and predicts $\hat{y} = \mathrm{Head}_{cls}(z_{x'_o})$. **Single-flow inference** uses the distilled anchor-only feature $\mathbf{f}_{\mathrm{single}}$ (§IV-D2) when neighbor flows are missing or unreliable. Padding is only used to keep a fixed input shape. TACL augmentations and AFP perturbations are disabled at inference. The anchor-only pathway retains context knowledge through HAD distillation during fine-tuning.

V. Experiment

A. Overview

In this section, we conduct empirical evaluations to demonstrate the effectiveness of the proposed Odysseus framework. Specifically, we aim to answer the following research questions:
RQ1: How does the traffic classification performance of Odysseus compare with SOTA baselines? (§V-C & §V-D)
RQ2: How do various key components in Odysseus contribute to its generalization performance? (§V-E)
RQ3: How does Odysseus perform in few-shot settings compared with strong baselines? (§V-F)
RQ4: What is the overhead of Odysseus? (§V-G)
RQ5: How does Odysseus perform under mixed-noise environments and compared with matrix-input-based baselines? (§V-H)

B. Experiment Setups

Datasets and evaluation protocol. We evaluate on four encrypted traffic datasets covering distinct deployment shifts: *software evolution* (APP53-Version [16]), *temporal drift* (APP53-Time [16]), *compound spatiotemporal shift* (APP60 [32]), and *intra-class heterogeneity* (ISCX-BotNet2014 [33]). We define IID (**In-***) and OOD (**Cross-***) settings sharing the same **source** training set. In **In-***, remaining source data is split into validation/test, and evaluation is on the *source test* split. In **Cross-***, remaining source data forms the validation set, and evaluation is on a disjoint **target** test set. To reduce correlation leakage from stride-1 windowing, splitting is performed before MSFGR construction, so windows derived from the same trace/capture file remain within the same split. Across all settings, model selection, hyperparameter tuning, and early stopping use only *source-validation* data, with no target-domain leakage. Unless specified, all *flow-level* results (including Odysseus (flow) and single-flow baselines) use the anchor flow at test time, with no neighbor flows. The multi-flow variant consumes the full context window.

Unlabeled pre-training corpus. We pre-train on ∼20 GB unlabeled traffic from (i) APP51, a private corpus of 51 popular mobile apps across social, video, shopping, music, local-service, and live-streaming categories, collected with the device/settings in [16], (ii) the APP53 *source* split (using only the source *training* portion), and (iii) CIC-IDS2017 [37]. APP51 is used only for label-free pre-training; the pre-training pipeline can be instantiated with public unlabeled traffic. A strict hold-out policy excludes downstream validation and test samples from pre-training.

APP53-Version / APP53-Time. Following [16], [18], APP53 contains encrypted traffic from popular mobile applications, collected across different time periods, app versions, and devices. We use its publicly available time- and version-shifted subsets to construct two evaluation protocols. **In-Version/Cross-Version** uses the source-version subset for training/validation and a disjoint subset captured after app version updates for testing, reflecting shifts caused by application updates. **In-Time/Cross-Time** uses source and target splits collected about one month apart, capturing temporal drift from evolving backends and deployment changes.

APP60. APP60 couples temporal evolution with spatial heterogeneity (e.g., ISP/network-condition changes). We use D1 as the source split and the strictly unseen D2 (collected several weeks later, with 44 updated apps) as the target split, and evaluate 56 classes (APP60 minus *Netflix*, *Piranhaoxue*, *WeLink*, and *WhatsApp*) to mitigate long-tail imbalance (**In-Spatiotemporal/Cross-Spatiotemporal**).

ISCX-BotNet2014. We restructure ISCX-BotNet2014 [33] (aggregating ISCX-2012 [38], ISOT [39], and MCFP) to study

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON RESULTS ON IID SCENARIOS. FOR EACH DATASET, **RANK** IS COMPUTED BY SORTING METHODS IN DESCENDING **F1** (1=BEST) AND IS SHOWN AS A SUPERSCRIPT NEXT TO **F1**. **TOTAL RANK** IS OBTAINED BY AVERAGING THE FIVE PER-DATASET F1-SCORE RANKS AND THEN RANKING METHODS BY THE AVERAGED RANK (LOWER IS BETTER). FOR EACH METRIC COLUMN, THE **BEST**/**SECOND-BEST**/THIRD-BEST METHOD IS HIGHLIGHTED.

Scenario (Dataset)	In-Version (APP53-Version)		In-Time (APP53-Time)		In-Spatiotemporal (APP60)		In-Type-2 (ISCX-BotNet2014)		In-Type-8 (ISCX-BotNet2014)		Total Rank
Method	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	
AppScanner	73.08	73.29[12]	72.65	72.27[11]	71.76	71.72[11]	99.54	99.53[6]	**95.60**	**95.36**[2]	9
CUMUL	61.00	60.22[13]	60.13	59.74[13]	68.08	67.91[12]	97.88	97.83[10]	86.86	87.78[6]	12
DF	73.44	73.75[11]	69.30	69.10[12]	65.80	65.84[13]	98.75	98.75[8]	78.78	75.06[12]	13
FS-Net	88.06	88.22[3]	85.75	85.56[6]	73.18	73.26[9]	98.13	98.12[9]	83.99	85.08[7]	7
PERT	76.40	76.77[10]	78.70	78.71[8]	75.61	72.29[10]	99.50	99.50[7]	76.03	72.29[13]	10
ET-BERT	87.82	87.94[5]	86.54	86.57[5]	92.93	92.91[3]	**99.92**	**99.92**[2]	77.03	81.48[10]	4
YaTC	87.78	88.08[4]	82.63	82.50[7]	**93.72**	**93.47**[2]	99.86	99.79[3]	93.22	92.61[3]	**2**
FG-Net	85.20	85.12[7]	89.50	89.20[3]	82.30	82.15[6]	99.65	99.60[5]	84.00	83.50[9]	6
TrafficScope	86.30	86.40[6]	87.20	87.10[4]	86.05	86.10[5]	95.30	95.20[13]	84.50	84.15[8]	8
FG-SAT	77.60	77.45[9]	77.10	76.80[10]	75.20	75.12[8]	97.50	97.40[11]	81.10	80.20[11]	11
SnifferDog	79.20	79.50[8]	78.10	78.40[9]	76.50	76.30[7]	**100.00**	**100.00**[1]	**99.88**	**99.83**[1]	5
ODYSSEUS (flow)	**96.92**	**96.93**[2]	**95.86**	**95.21**[2]	86.33	86.46[4]	96.63	96.62[12]	90.24	90.12[4]	3
ODYSSEUS (context)	**99.78**	**99.78**[1]	**99.67**	**99.61**[1]	**96.42**	**96.36**[1]	99.75	99.75[4]	93.83	89.69[5]	**1**

shifts from changing botnet-family mixtures. We evaluate **In-Type-2/Cross-Type-2** (Benign vs. Botnet) with two OOD levels: *Weak* uses the same botnet families as training but different samples, while *Strong* tests on a broader mixture that covers all available botnet types (including under-represented ones). We also evaluate **In-Type-8/Cross-Type-8** over {Benign, Neris, NSIS, RBot, SMTP_Spam, Virut, Zeus, Zeus_control}, where the target split comes from different sources/timeframes than the source split.

Baselines. We compare against SOTA methods across routes: traditional ML (AppScanner [22], CUMUL [23]), deep learning (DF [24], FS-Net [26]), pre-training (PERT [13], ET-BERT [14], YaTC [15]), multi-flow (FG-Net [16], TrafficScope [17]), and OOD/robustness-oriented methods (FG-SAT [5], SnifferDog [21], ETooL [18], OWAD [40]).

Implementation. Experiments run on NVIDIA Tesla V100S GPUs using AdamW. Pre-training uses batch size 64 and effective LR 1.5×10^{-3} for 50 epochs. Fine-tuning uses batch size 32, dropout 0.1, and LR 2×10^{-3} for up to 70 epochs. For both IID (§V-C) and OOD (§V-D) tasks, we report testing performance using the checkpoint that achieves the highest *source*-validation F1-score (strictly no target-domain model selection). We report sample-level accuracy (AC) and macro-averaged precision (PR), recall (RC), and F1-score (F1) for all classification results.

C. Evaluation in IID Scenarios (RQ1)

As shown in Table I, ODYSSEUS (context) achieves the best overall rank in IID evaluation, reaching 99.78/99.61/96.36 F1-Score on In-Version/In-Time/In-Spatiotemporal. The gains are most pronounced on application identification: on In-Version, it improves F1-Score over AppScanner, FS-Net, and ET-BERT by 26.49/11.56/11.84%, on In-Time, the gains are 27.34/14.05/13.04%. This indicates that MSFGR-based multi-flow context provides more stable behavior-level cues, and context-level pre-training further strengthens them.

Among pre-training baselines, YaTC is the strongest single-flow method, but ODYSSEUS further benefits from context-level MSFGR and HAD, yielding strong context inference and competitive anchor-only inference.

On botnet detection, ODYSSEUS is near-ceiling on Type-2 and strong on Type-8, while SnifferDog's perfect In-Type-2 F1-score aligns with IP-centric memorization under IID splits. ODYSSEUS (flow) remains competitive on app identification (top-2/top-2/top-4 on In-Version/In-Time/In-Spatiotemporal), validating that distillation preserves much of the context benefit.

D. Evaluation in OOD Scenarios (RQ1)

Table II summarizes OOD results under three application shifts (Cross-Version/Time/Spatiotemporal) and three botnet shifts (Cross-Type-2 Weak/Strong and Cross-Type-8). ODYSSEUS (context) ranks 1st overall, achieving 85.38/90.05/71.93 F1-Score on Cross-Version/Cross-Time/Cross-Spatiotemporal and 99.62/96.87/72.55 F1-Score on the three Cross-Type scenarios. On application recognition, it improves F1 over YaTC by +12.30 (Cross-Version) and +15.70 (Cross-Time), suggesting that single-flow representations are still limited under logic-driven shifts even with self-supervised pre-training.

This gap reflects inductive biases: AppScanner depends on limited flow statistics, FS-Net overfits single-flow sequences, while FG-Net/TrafficScope lack context-level pre-training, task-driven regularization, and hierarchical distillation.

On botnet shifts, ODYSSEUS is best on Cross-Type-2 Weak and is stronger than SnifferDog on Cross-Type-2 Strong (96.87 vs. 94.87). For Cross-Type-8, ETooL is best while ODYSSEUS is second-best. ODYSSEUS (flow) stays strong (overall rank 4), indicating HAD distillation retains much of the context robustness for single-flow inference.

E. Ablation Study (RQ2)

a) Ablation settings and discussion: As shown in Table III, we systematically ablate key components. Removing intra-flow attention (w/o Intra-Flow Attn) eliminates per-flow encoding, causing significant context degradation. Removing inter-flow attention (w/o Inter-Flow Attn) deletes cross-flow dependency

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

TABLE II

MERGED COMPARISON RESULTS ON OOD SCENARIOS, FOLLOWING THE SAME RANKING/HIGHLIGHTING PROTOCOL AS TABLE I. SOME BASELINES ARE NOT APPLICABLE TO ALL SCENARIOS DUE TO TASK/DATASET CONSTRAINTS, AND WE REPORT THEIR RESULTS WHERE THE ORIGINAL METHOD SUPPORTS THE CORRESPONDING SETTING.

Scenario (Dataset)	Cross-Version (APP53-Version)		Cross-Time (APP53-Time)		Cross-Spatiotemporal (APP60)		Cross-Type-2 Weak (ISCX)		Cross-Type-2 Strong (ISCX)		Cross-Type-8 (ISCX-BotNet2014)		Total Rank
Method	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	**AC%**	**F1%**	
AppScanner	35.39	35.00[13]	43.87	44.03[11]	56.89	51.87[7]	80.99	80.98[13]	76.94	76.92[14]	60.99	60.16[10]	11
CUMUL	36.83	36.14[12]	37.78	37.46[13]	42.74	38.87[13]	60.08	58.70[14]	65.95	65.42[15]	29.41	28.37[14]	13
DF	34.58	33.78[14]	34.56	34.56[14]	42.88	42.70[12]	84.50	84.60[10]	78.25	77.94[12]	26.70	42.30[13]	12
FS-Net	43.14	43.48[11]	45.24	43.96[12]	57.21	52.41[5]	83.72	84.49[11]	77.42	77.27[13]	70.33	69.87[4]	10
PERT	45.96	45.72[10]	45.69	46.18[10]	51.04	50.08[8]	93.00	93.00[8]	85.50	85.20[8]	65.38	61.48[9]	8
ET-BERT	57.08	56.98[5]	57.04	56.71[5]	52.54	52.20[6]	96.50	96.50[3]	85.75	85.48[7]	65.51	67.50[5]	3
YaTC	**74.04**	**73.08**[2]	**74.50**	**74.35**[2]	**60.96**	**60.56**[2]	96.13	96.12[4]	89.76	90.15[5]	68.69	65.72[6]	**2**
FG-Net	54.10	53.40[9]	51.80	51.20[8]	49.10	48.50[10]	91.40	91.20[9]	85.30	85.10[9]	59.50	58.20[11]	9
TrafficScope	55.40	55.10[7]	53.20	52.80[7]	50.10	49.60[9]	95.60	95.50[5]	91.60	91.40[4]	62.50	62.15[8]	7
FG-SAT	59.00	58.15[4]	60.10	59.40[4]	53.50	52.80[4]	93.60	93.50[7]	84.80	84.60[10]	64.10	63.50[7]	6
SnifferDog	54.50	54.20[8]	50.50	50.10[9]	48.00	47.30[11]	**99.59**	**99.60**[2]	95.15	94.87[3]	**75.16**	72.10[3]	5
ETooL	57.08	56.98[5]	57.04	56.71[5]	–	–	–	–	**95.36**	**95.03**[2]	**84.19**	**81.95**[1]	–
OWAD	–	–	–	–	–	–	80.75	83.65[12]	77.12	81.27[11]	–	–	–
ODYSSEUS (flow)	60.52	59.61[3]	60.26	60.55[3]	53.83	53.35[3]	94.75	94.74[6]	87.88	87.96[6]	56.89	56.41[12]	4
ODYSSEUS (context)	**85.76**	**85.38**[1]	**91.20**	**90.05**[1]	**72.28**	**71.93**[1]	**99.62**	**99.62**[1]	**96.88**	**96.87**[1]	71.09	**72.55**[2]	**1**

modeling, reducing both context and distilled single-flow performance. Replacing attention-guided pooling with weaker alternatives (Global Avg, CLS-only) degrades performance, confirming the importance of selecting informative patches. Disabling AFP (loss weight=0) removes semantic-consistency supervision, causing clear performance drops. Disabling TACL (contrastive loss=0) slightly improves single-flow metrics but reduces context robustness. w/o Distill (distillation loss=0) causes consistent single-flow performance drops, while w/o Pre-train (random initialization) substantially harms both branches. Removing all modules leads to the largest performance drop, confirming the effectiveness of our multi-level self-attention modeling and pre-training for OOD generalization.

b) Impact of Pre-training: We further compare the pre-trained model against training from scratch (settings with and without pre-training, as shown in Fig. 3). Pre-training not only improves final test performance but also accelerates optimization: the pre-trained variant requires markedly fewer epochs to reach high F1-Score, indicating faster and more sample-efficient adaptation. Over the full training horizon, the pre-trained model maintains higher F1 trajectories for both branches. This advantage is accompanied by more stable optimization: both training and validation losses decrease faster and remain lower with pre-training, suggesting improved conditioning and reduced reliance on brittle shortcut cues during fine-tuning.

F. Few-shot Study (RQ3)

To evaluate OOD generalization with limited labeled data, we use the Cross-Spatiotemporal scenario, vary the labeled fraction of the source training split (10%, 50%, and 100%), and evaluate all methods on the fixed target split. As shown in Fig. 4, pre-trained models, including PERT, ET-BERT, YaTC, and ODYSSEUS, show smaller performance declines as labeled data decreases. Pre-training helps extract high-quality representations from unlabeled data, thereby reducing dependence on labeled data. Moreover, ODYSSEUS consistently outperforms single-flow pre-trained baselines, indicating that short-window multi-flow context provides complementary behavioral cues under label scarcity. This suggests that context-level pre-training is especially beneficial when labeled source data is limited.

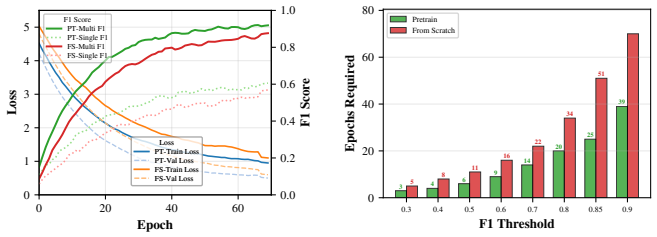
TABLE III

ABLATION RESULTS ON THE CROSS-TIME DATASET. VALUES IN PARENTHESES DENOTE CHANGES RELATIVE TO THE FULL MODEL.

No.	Multi-Acc	Multi-F1	Single-Acc	Single-F1
0	91.20	90.05	60.26	60.55
1	84.39 (-6.81)	83.20 (-6.85)	84.39†	83.20†
2	89.09 (-2.11)	88.00 (-2.05)	59.00 (-1.26)	58.60 (-1.95)
3	90.69 (-0.51)	89.60 (-0.45)	60.70 (+0.44)	60.80 (+0.25)
4	90.09 (-1.11)	88.90 (-1.15)	59.20 (-1.06)	58.90 (-1.65)
5	88.59 (-2.61)	87.40 (-2.65)	59.09 (-1.17)	58.95 (-1.60)
6	89.80 (-1.40)	89.10 (-0.95)	60.80 (+0.54)	60.90 (+0.35)
7	90.30 (-0.90)	89.20 (-0.85)	57.80 (-2.46)	57.40 (-3.15)
8	82.89 (-8.31)	81.40 (-8.65)	53.59 (-6.67)	53.10 (-7.45)
9	70.20 (-21.00)	70.05 (-20.00)	70.20†	70.05†

Serial-number settings: 0: Full model. 1: w/o Intra-Flow Attn. 2: w/o Inter-Flow Attn. 3: w/o Attn-Guided Pooling (Global Avg). 4: w/o Attn-Guided Pooling (CLS-only). 5: w/o AFP. 6: w/o TACL. 7: w/o Distill. 8: w/o Pretrain. 9: w/o All

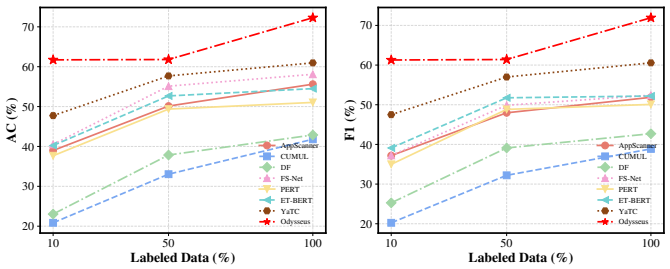
† Single-flow branch is not applicable in these degenerated settings; the reported values coincide with the context branch.



(a) F1 Comparison & Loss Curves (b) Convergence Speed

Fig. 3. Comparison of fine-tuning performance based on pre-train and from scratch on Cross-Time.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.



(a) Accuracy (b) F1-Score

Fig. 4. Few-shot performance comparison on APP60.

TABLE IV
STUDY ON THE TIME AND SPACE EFFICIENCY.[2]

ID	Train Time	Tuned Params	GPU (MB)	FLOPs (G)	Latency (ms)
1	2:47:53	1,250,100	N/A	0.000016	55.66
2	2:02:35	3,761,275	10,598	0.165205	37.09
3	0:35:43	132,146,715	20,364	12.698963	2.20
4	-	131,612,672	71,196	2.793692	3,756.62
5	1:41:34	1,865,503	10,157	1.781022	0.69

G. Efficiency Study (RQ4)

We benchmark ODYSSEUS against representative baselines—AppScanner (classical ML), FS-Net (sequential DL), ET-BERT (pre-trained), and ETooL (LLM-based OOD)—using directly measured results for reproducible baselines and reported implementation-level statistics for ETooL (Table IV). ODYSSEUS offers a favorable efficiency–accuracy trade-off: it tunes only 1.87M parameters, uses 10.2 GB GPU memory, and achieves 0.69 ms latency while maintaining strong prediction quality. In contrast, AppScanner is CPU-oriented, FS-Net's recurrent dependencies limit parallelism, ET-BERT requires a much larger pre-trained backbone, and ETooL incurs much higher memory and response latency due to its LLM-based pipeline. Overall, ODYSSEUS achieves strong OOD performance with practical runtime and memory overhead.

H. Deep Dive Analysis (RQ5)

1) Context Robustness under Noise: As shown in Table V, ODYSSEUS degrades gracefully as more neighbor flows are replaced by other-class flows, and remains stronger than the distilled flow-only branch under both Weak and Strong settings. Even when 75% of neighbor flows are replaced, the context branch still achieves 95.87% and 89.51% F1 on Weak and Strong, respectively, outperforming the flow-only branch by 1.13% and 1.55%. A qualitative error inspection further suggests that noisy context mainly increases false negatives while keeping false positives low, indicating conservative degradation rather than excessive false alarms. This behavior is consistent with our design: MSFGR preserves robust intra-flow evidence, while AFP encourages the model to detect semantic inconsistency under different-label flow substitutions instead of blindly relying on all neighboring flows. The size-3 context and flow-only results further show that ODYSSEUS remains effective when context is incomplete or unavailable.

[2]Method IDs: (1) AppScanner, (2) FS-Net, (3) ET-BERT, (4) ETooL (Traffic Task Freeze), and (5) ODYSSEUS. FLOPs are reported in GFLOPs. FS-Net, ET-BERT, and ODYSSEUS are evaluated on Cross-Time using the same Tesla V100 GPU and batch size 128. AppScanner is CPU-oriented, so GPU memory is N/A. ETooL training time is omitted due to different instruction-tuning data and hardware settings; other ETooL statistics follow the authors' reported Traffic Task Freeze configuration.

TABLE V
CROSS-TYPE-2 PERFORMANCE UNDER VARYING CONTEXT SIZES AND TEST-TIME CONTEXT NOISE. NOISE IS INJECTED BY RANDOMLY REPLACING k NEIGHBOR FLOWS DURING INFERENCE ON THE IDENTICALLY TRAINED MODEL.

Scenario	Cross-Type-2-Weak				Cross-Type-2-Strong			
Setting	AC%	PR%	RC%	F1%	AC%	PR%	RC%	F1%
k0 (0% noise)	99.62	99.63	99.63	99.62	96.88	96.89	96.88	96.87
k1 (25% noise)	98.50	98.53	98.50	98.50	94.88	95.22	94.88	94.87
k2 (50% noise)	97.12	97.26	97.12	97.12	92.38	93.38	92.38	92.33
k3 (75% noise)	95.88	96.15	95.88	95.87	89.62	91.41	89.62	89.51
Context (size=3)	98.80	98.85	98.80	98.80	95.40	95.45	95.40	95.40
ODYSSEUS (flow)	94.75	94.73	94.75	94.74	87.88	88.04	87.88	87.96

TABLE VI
COMPARISON RESULTS OF MATRIX-METHODS ON CROSS-VERSION AND CROSS-TIME

Dataset	Cross-Version				Cross-Time			
Method	AC%	PR%	RC%	F1%	AC%	PR%	RC%	F1%
mini-FlowPic	14.98	14.18	15.39	11.68	12.70	13.16	12.70	11.25
2D-CNN	56.22	55.14	56.05	55.59	57.70	57.73	57.95	57.84
MarkovGAN	62.32	62.64	62.10	62.37	54.00	58.42	54.35	56.31
ODYSSEUS (flow)	60.52	58.94	60.30	59.61	60.26	61.01	60.10	60.55
ODYSSEUS (context)	**85.76**	**85.21**	**85.55**	**85.38**	**91.20**	**90.32**	**90.01**	**90.05**

2) Comparison within Matrix-based Methods: We compare ODYSSEUS with representative matrix-based pipelines, including mini-FlowPic [35], 2D-CNN [41], and MarkovGAN [42]. As shown in Table VI, ODYSSEUS achieves the best OOD performance on both Cross-Version and Cross-Time by modeling intra-/inter-flow structure in MSFGR and distilling contextual cues into a deployable single-flow branch, whereas prior matrix encodings mainly capture sparse length–time bins, byte grids, or local transition patterns.

VI. CONCLUSION

We propose ODYSSEUS, a context-level pre-training framework for OOD encrypted traffic classification. By combining MSFGR, dual-level masked pre-training, HAD, TACL, and AFP, ODYSSEUS learns transferable intra-/inter-flow representations and supports both context-level and flow-level inference. Extensive experiments across 11 IID/OOD scenarios demonstrate its state-of-the-art performance, few-shot robustness, graceful degradation under noisy/missing context, and practical efficiency. These results suggest that short-window multi-flow context, when coupled with self-supervised pre-training and context-to-flow transfer, can serve as a practical path toward robust encrypted traffic classification under distribution shifts.

ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (No. 2024YFF1401300).

REFERENCES

[1] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 791–824, 2023.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.

[2] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–35, 2021.

[3] W. Dong, J. Yu, X. Lin, G. Gou, and G. Xiong, "Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review," *Neurocomputing*, p. 128444, 2024.

[4] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.

[5] S. Cui, X. Han, D. Han, Z. Wang, W. Wang, B. Jiang, B. Liu, and Z. Lu, "Fg-sat: Efficient flow graph for encrypted traffic classification under environment shifts," *IEEE Transactions on Information Forensics and Security (TIFS)*, 2025.

[6] Y. Qing, Q. Yin, X. Deng, X. Zhang, P. Li, Z. Liu, K. Sun, K. Xu, and Q. Li, "Training robust classifiers for classifying encrypted traffic under dynamic network conditions," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, pp. 3564–3578.

[7] P. Liu, L. Li, Y. Yan, M. Fazzini, and J. Grundy, "Identifying and characterizing silently-evolved methods in the android api," in *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2021, pp. 308–317.

[8] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[9] M. Jiang, M. Cui, C. Liu, G. Gou, G. Xiong, and Z. Li, "Zero-relabelling mobile-app identification over drifted encrypted network traffic," *Computer Networks*, vol. 228, p. 109728, 2023.

[10] N. Wickramasinghe, A. Shaghaghi, G. Tsudik, and S. Jha, "Sok: Decoding the enigma of encrypted network traffic classifiers," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1825–1843.

[11] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[12] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 16 000–16 009.

[13] H. He, Z. Yang, and X. Chen, "Pert: Payload encoding representation from transformer for encrypted traffic classification," in *Proceedings of the ITU Kaleidoscope (ITU K)*, 2020, pp. 1–8.

[14] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM International Conference on World Wide Web (WWW)*, 2022, pp. 633–642.

[15] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, and Z. Xue, "Yet another traffic classifier: a masked autoencoder based traffic transformer with multi-level flow representation," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2023, pp. 5420–5427.

[16] M. Jiang, Z. Li, P. Fu, W. Cai, M. Cui, G. Xiong, and G. Gou, "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Computer Networks*, vol. 217, p. 109309, 2022.

[17] Z. Zhao, Z. Song, X. Xie, Z. Li, J. Yu, F. Zhang, and T. Li, "Towards context-aware traffic classification via time-wavelet fusion network," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025, pp. 2089–2100.

[18] X. Lin, G. Xiong, G. Gou, W. Dong, J. Yu, Z. Li, and W. Xia, "Respond to change with constancy: Instruction-tuning with llm for non-iid network traffic classification," *IEEE Transactions on Information Forensics and Security (TIFS)*, 2025.

[19] R. Zhao, X. Deng, Y. Wang, L. Chen, M. Liu, Z. Xue, and Y. Wang, "Flow sequence-based anonymity network traffic identification with residual graph convolutional networks," in *Proceedings of the IEEE/ACM International Symposium on Quality of Service (IWQoS)*, 2022, pp. 1–10.

[20] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "Mt-flowformer: A semi-supervised flow transformer for encrypted traffic classification," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 2576–2584.

[21] X. Luo, L. Yin, H. Yang, Z. Liu, W. Chen, S. Jia, B. Luo, and H. Xiang, "Snifferdog: Comprehensively learning heterogeneous features of network traffic to identify malicious flows," *IEEE Transactions on Information Forensics and Security (TIFS)*, 2025.

[22] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 439–454.

[23] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale," in *Proceedings of the Network and distributed system security symposium (NDSS)*, 2016.

[24] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.

[25] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2017, pp. 43–48.

[26] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "Fs-net: A flow sequence network for encrypted traffic classification," in *Proceedings of the IEEE International Conference On Computer Communications (INFOCOM)*, 2019, pp. 1171–1179.

[27] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, and B. Thuraisingham, "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proceedings of the Annual Conference on Computer Security Applications (ACSAC)*, 2016, pp. 177–188.

[28] R. Attarian, L. Abdi, and S. Hashemi, "Adawfpa: Adaptive online website fingerprinting attack for tor anonymous network: A stream-wise paradigm," *Computer Communications*, vol. 148, pp. 74–85, 2019.

[29] L. Cai, X. Jing, S. Sun, H. Huang, N. Chen, and Y. Lu, "P2p traffic identification based on transfer learning," in *Proceedings of the IEEE International Conference on Granular Computing*, 2013, pp. 22–26.

[30] G. Sun, L. Liang, T. Chen, F. Xiao, and F. Lang, "Network traffic classification based on transfer learning," *Computers & Electrical Engineering*, vol. 69, pp. 920–927, 2018.

[31] S. Zhao, M. Gong, T. Liu, H. Fu, and D. Tao, "Domain generalization via entropy regularization," in *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, 2020, pp. 16 096–16 107.

[32] J. Shi, M. Liu, C. Hou, M. Jiang, and G. Xiong, "Online encrypted mobile application traffic classification at the early stage: Challenges, evaluation criteria, comparison methods," in *Proceedings of the IEEE 6th International Conference on Computer and Communication Systems (ICCCS)*, 2021, pp. 1128–1135.

[33] E. B. Beigi, H. H. Jazi, N. Stakhanova, and A. A. Ghorbani, "Towards effective feature selection in machine learning-based botnet detection approaches," in *Proceedings of the IEEE Conference on Communications and Network Security (CNS)*, 2014, pp. 247–255.

[34] J. Wang, C. Lan, C. Liu, Y. Ouyang, T. Qin, W. Lu, Y. Chen, W. Zeng, and P. S. Yu, "Generalizing to unseen domains: A survey on domain generalization," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 35, no. 8, p. 8052–8072, 2023.

[35] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-flowpic augmentations," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.

[36] M. Yi, L. Hou, J. Sun, L. Shang, X. Jiang, Q. Liu, and Z. Ma, "Improved ood generalization via adversarial training and pretraining," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 11 987–11 997.

[37] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani *et al.*, "Toward generating a new intrusion detection dataset and intrusion traffic characterization." *ICISSP*, vol. 1, no. 2018, pp. 108–116, 2018.

[38] A. Shiravi, H. Shiravi, M. Tavallaee, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Computers & Security*, vol. 31, no. 3, pp. 357–374, 2012.

[39] D. Zhao, I. Traore, B. Sayed, W. Lu, S. Saad, A. Ghorbani, and D. Garant, "Botnet detection based on traffic behavior analysis and flow intervals," *Computers & Security*, vol. 39, pp. 2–16, 2013.

[40] D. Han, Z. Wang, W. Chen, K. Wang, R. Yu, S. Wang, H. Zhang, Z. Wang, M. Jin, J. Yang *et al.*, "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation." in *NDSS*, 2023.

[41] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *IEEE International Conference on Information Networking*, 2017.

[42] Z. Tang, J. Wang, B. Yuan, H. Li, J. Zhang, and H. Wang, "Markov-gan: Markov image enhancement method for malicious encrypted traffic classification," *IET Information Security*, vol. 16, no. 6, pp. 442–458, 2022.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:33:36 UTC from IEEE Xplore. Restrictions apply.